ASSIGNMENT-10.3

HT.NO-2403A52L06

BATCH-50

Task-1:

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to

calculate factorials: def factorial(n):

result = 1 for i in

range(1, n): result

= result * i

return result

Instructions:

1. Run the code and test it with factorial(5).

2. Use an AI assistant to:

o Identify the logical bug in the code.

o Explain why the bug occurs (e.g., off-by-one error).

o Provide a corrected version.

3. Compare the AI's corrected code with your own manual fix. 4.

Write a brief comparison: Did AI miss any edge cases (e.g.,

negative numbers, zero)?

Expected Output:

Corrected function should return 120 for factorial(5)

PROMPT:

There is a given code available and there is a logical error in that code fix the bug , explain the why the bug occurs and also

Provide correct code

CODE:

```
# Original buggy function def
factorial_buggy(n):
    result = 1
    for i in range(1, n):  # BUG: range(1, n) excludes n
result = result * i     return result

# Corrected function def
factorial(n):
    """Calculate factorial of n."""
if n < 0:
        raise ValueError("Factorial not defined for negative numbers")
if n == 0 or n == 1:
        return 1     result = 1
    for i in range(1, n + 1):  # FIX: range(1, n+1) includes n
result = result * i     return result

# Test print("Problem 1 - Factorial:")
print(f"factorial(5) = {factorial(5)}")
```

TASK-2:

Problem Statement 2: Task 2 — Improving Readability & Documentation

Scenario:The following code works but is poorly written:

.

```
def calc(a, b, c):
if c == "add":
return a + b elif
c == "sub":
return a - b elif
```

c == "mul":

return a * b

elif c == "div":

Instructions:

5. Use AI to: o Critique the function’s readability,

parameter naming, and lack of documentation. o Rewrite

the function with:

1. Descriptive function and parameter names.
2. A complete docstring (description, parameters, return value, examples).
3. Exception handling for division by zero.
4. Consideration of input validation.

6. Compare the original and AI-improved versions.

7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).

Expected Output:

A well-documented, robust, and readable function that handles

errors gracefully.

CODE:

```
def perform_arithmetic_operation(operand1, operand2, operation):
    """
    Perform arithmetic operations on two operands.

    Parameters:
        operand1 (float): The first operand.        operand2 (float): The
second operand.        operation (str): The operation to perform ('add',
'sub', 'mul', 'div').

    Returns:
```

```
        float: The result of the operation.


    Raises:
        ValueError: If operation is invalid or division by zero occurs.


    Examples:
        >>> perform_arithmetic_operation(10, 5, 'add')
        15
        >>> perform_arithmetic_operation(10, 5, 'div')
        2.0     """     if
operation == "add":
        return operand1 + operand2
elif operation == "sub":
        return operand1 - operand2
elif operation == "mul":
        return operand1 * operand2
elif operation == "div":         if
operand2 == 0:
            raise ValueError("Division by zero is not allowed")
return operand1 / operand2
    else:
        raise ValueError(f"Unknown operation: {operation}")

# Test print("\nProblem 2 - Arithmetic
Operations:")
print(f"perform_arithmetic_operation(10, 5, 'add') = {perform_arithmetic_operation(10, 5, 'add')}")
print(f"perform_arithmetic_operation(10, 5, 'div') = {perform_arithmetic_operation(10, 5, 'div')}")
```

TASK-3:

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

def Checkprime(n):

for i in range(2, n):

if n % i == 0: return

False return True

Instructions:

8. Verify the function works correctly for sample inputs.

9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to: o List all PEP8 violations.

o Refactor the code (function name, spacing, indentation, naming).

10. Apply the AI-suggested changes and verify functionality is preserved.

11. Write a short note on how automated AI reviews could streamline code reviews in large teams.

Expected Output:

A PEP8-compliant version of the function, e.g.:

def check_prime(n): for i in range(2, n):

if n % i == 0:

return False

return True

CODE:

```
def check_prime(n):
    """
    Check if a number is prime.

    Parameters:
        n (int): The number to check.
```

```
    Returns:
        bool: True if n is prime, False otherwise.
    """     if
n < 2:
        return False
for i in range(2, n):
if n % i == 0:
return False     return
True


# Test print("\nProblem 3 - Prime Checker:")
print(f"check_prime(7) = {check_prime(7)}")  # Expected: True
print(f"check_prime(10) = {check_prime(10)}")  # Expected: False
```

TASK-4:

Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

```
def processData(d):
return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

1. Manually review the function for:

o Readability and naming. o Reusability and modularity.

o Edge cases (non-list input, empty list, non-integer elements).

2. Use AI to generate a code review covering:

a. Better naming and function purpose clarity.

b. Input validation and type hints.

c. Suggestions for generalization (e.g., configurable multiplier).

3. Refactor the function based on AI feedback.

4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output:

An improved function with type hints, validation, and clearer intent, e.g.:

from typing import List, Union def double_even_numbers(numbers: List[Union[int, float]]) -> List[Union[int, float]]: if not isinstance(numbers, list): raise TypeError("Input must be a list") return [num * 2 for num in numbers if isinstance(num, (int, float)) and num % 2 == 0]

CODE:

```
def double_even_numbers(numbers: List[Union[int, float]]) -> List[Union[int, float]]:
    """
    Double all even numbers in a list.

    Parameters:
        numbers (List[Union[int, float]]): A list of integers or floats.

    Returns:
        List[Union[int, float]]: A new list with even numbers doubled.

    Raises:
        TypeError: If input is not a list.
    """     if not
isinstance(numbers, list):
```

raise TypeError("Input must be a list") return [num * 2 for num in numbers if isinstance(num, (int, float)) and num % 2 == 0]

Test print("\nProblem 4 - Double Even Numbers:") print(f"double_even_numbers([1, 2, 3, 4, 5]) = {double_even_numbers([1, 2, 3, 4, 5])}")

TASK-5:

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets: def sum_of_squares(numbers):

total = 0 for num in numbers: total += num ** 2 return total

Instructions:

1. Test the function with a large list (e.g., range(1000000)).
2. Use AI to:
   - o Analyze time complexity.
   - o Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable). o Provide an optimized version.
3. Compare execution time before and after optimization.
4. Discuss trade-offs between readability and performance.

Expected Output: An optimized function, such as: def sum_of_squares_optimized(numbers):

return sum(x * x for x in numbers)

CODE:

```
def sum_of_squares(numbers):
    """Calculate sum of squares (basic implementation)."""
```

```
    total = 0     for num
in numbers:         total
+= num ** 2     return
total

def sum_of_squares_optimized(numbers):
    """Calculate sum of squares (optimized implementation)."""
return sum(x * x for x in numbers)

# Test with large dataset print("\nProblem 5 -
Performance Optimization:")

large_list = list(range(1000000))

start = time.time()
result1 = sum_of_squares(large_list) time1
= time.time() - start

start = time.time() result2 =
sum_of_squares_optimized(large_list) time2 =
time.time() - start

print(f"Both functions return same result: {result1 == result2}")
print(f"Basic version time: {time1:.6f}s") print(f"Optimized
version time: {time2:.6f}s") print(f"Speedup:
{time1/time2:.2f}x")
```

OUTPUT:

PROBLEMS 533 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS Python

```
sanjaykumar@perumandlas-MacBook-Air AI - assiant %  source "/Users/sanjaykumar/Documents/AI - assiant/.venv/bin/acti
vate"
"/Users/sanjaykumar/Documents/AI - assiant/.venv/bin/python" "/Users/sanjaykumar/Documents/AI - assiant/ass_10.py"
(.venv) sanjaykumar@perumandlas-MacBook-Air AI - assiant % "/Users/sanjaykumar/Documents/AI - assiant/.venv/bin/pyth
on" "/Users/sanjaykumar/Documents/AI - assiant/ass_10.py"
Problem 1 - Factorial:
factorial(5) = 120

Problem 2 - Arithmetic Operations:
perform_arithmetic_operation(10, 5, 'add') = 15
perform_arithmetic_operation(10, 5, 'div') = 2.0

Problem 3 - Prime Checker:
check_prime(7) = True
check_prime(10) = False

Problem 4 - Double Even Numbers:
double_even_numbers([1, 2, 3, 4, 5]) = [4, 8]

Problem 5 - Performance Optimization:
Both functions return same result: True
Basic version time: 0.029120s
Optimized version time: 0.024565s
Speedup: 1.19x
(.venv) sanjaykumar@perumandlas-MacBook-Air AI - assiant %
```